Technical overview — accessible HTML from PowerPoint data-structure animations

What it does

A command-line Java tool that ingests a `.pptx`, detects each tree diagram, reconstructs its logical structure from the slide geometry, and emits one self-contained, screen-reader-navigable HTML page per diagram. Target users: blind and low-vision students using the NVDA screen reader on Windows.

```
java -jar pptx-accessibility.jar lecture.pptx outputDir
```

Why it's non-trivial

A `.pptx` is an OOXML package — a zip of XML parts. Crucially, a slide stores *shapes with coordinates*, not *relationships*. The file records "an ellipse labelled G at (x, y)" and "a line with these endpoints," but nowhere does it say that G is the parent of C. The hierarchy exists only implicitly, in the spatial layout and the animation timeline. The core task is therefore to **recover that implicit structure from geometry** and re-express it as keyboard-navigable structured text — the same approach shown to work for accessible mathematics (MathML + the Speech Rule Engine), applied here to animated data-structure diagrams.

Pipeline

The type handed between each stage is shown in brackets.

```
lecture.pptx                                          [OOXML zip]
  → PptxAccessibilityTool : unzip, resolve true slide order, read slide size
  → SlideParser.parse      → classified shapes          [List<AvShape>]
    SlideParser.parseSteps → per-click motion deltas     [List<List<Move>>]
  → route on shape composition:
      table node-boxes  → BTreeBuilder → insertion states  [List<State>]
      ellipse nodes     → TreeBuilder  → trees             [List<Diagram>]
      picture only      → skip + warn
  → HtmlGenerator         → one self-contained page per diagram   [.html]
```

In one line: unzip → classify shapes per slide → detect structure type → reconstruct topology from geometry → render as an accessible page.

The classes

Class	Responsibility	Key technique
`PptxAccessibilityTool`	Orchestration and I/O	Resolves real slide order from `presentation.xml` `sldIdLst` (relationship-resolved), not file-name order; routes each slide by shape composition
`Xml`	DOM traversal helpers	Matches elements by *local* name, sidestepping the `p:` / `a:` / `r:` namespace prefixes
`SlideParser`	Slide XML → model	Classifies each shape (NODE / EDGE / NODEBOX / KEY / OTHER), converts geometry EMU→px, and extracts the animation main sequence into per-click motion deltas
`AvShape`	Shape value object	Holds geometry, text, table cells, flip flags, and the assigned kind
`TreeBuilder`	Binary / BST / AVL reconstruction	Edge→nearest-node matching (flip-corrected endpoints, distance-thresholded); union-find groups joined nodes into separate trees; parent = lesser *y*, left/right by *x*
`TreeNode`	Binary node	value, left/right/parent, centre coordinates
`BTreeBuilder`	B-tree reconstruction	Applies motion deltas to get resting positions; node keys = table cells ∪ contained key-boxes (x-sorted, deduped); boxes linked by lines → connected components = per-insertion states, ordered by *y*; inserted key found by multiset diff of consecutive states
`BNode`	B-tree node	ordered keys, ordered children
`HtmlGenerator`	Render	Emits self-contained HTML driven by a single ARIA live region; a `keydown` handler moves a cursor and rewrites the region; tree is serialised inline as a JS object

Points worth highlighting (and likely questions)

Structure is inferred, not read. The whole TreeBuilder / BTreeBuilder layer exists because topology is never stored — it is derived from coordinates and the animation. This is the conceptual heart of the project.

Coordinates are EMUs. OOXML positions are in English Metric Units: 914,400 per inch, i.e. 9,525 per pixel. Every coordinate is divided by 9,525 on the way in.

Animation is relative motion. A moving shape stores an `animMotion` *path* in slide-fraction units, e.g. `L 0.126 0.002` = move 12.6% of slide width, 0.2% of height. Resting position = authored position + (path fraction × slide size). The tool keeps the final `L` of the path (the

end offset) and scales it to pixels.

B-tree slides are a vertical filmstrip. They are not an in-place animation; each insertion is drawn as a separate snapshot stacked down the slide, with the new key sliding in from the margin. Each snapshot is recovered as a connected component of boxes; ordering the snapshots top-to-bottom yields the insertion sequence. Node contents come from the `<a:tbl>` cells merged with any key-boxes that slid on top.

Single output channel. The page exposes one `aria-live="assertive"`, `aria-atomic="true"` region. Every keypress moves a cursor and rewrites that region, so NVDA announces the new state immediately with no focus change. Because ARIA live regions and `keydown` are standards-based, the same page also works in JAWS. Keyboard model: binary trees use parent / left / right / root; B-trees use parent / first-child / previous-sibling / next-sibling; Page Up/Down step through insertions. Each file is standalone (inline CSS + JS) and works offline.

No Apache POI. Parsing uses only `java.util.zip` + `javax.xml`, so there are no third-party dependencies to install — important for locked-down institutional or lab machines.

Routing logic (and a known limitation)

Per slide, in this order:

Condition	Treated as	Output
`any table node-box present`	B-tree	`slideN_btree.html`
`else any ellipse node present`	binary tree	`slideN_diagramM.html` (one per tree)
`else a picture present`	image-only	skipped, warning printed

Limitation: the first test (any table → B-tree) wins, so a slide carrying even one stray table is routed to the B-tree path and its ellipse nodes are ignored. The sturdier rule keys off the slide title ("Binary Search Tree" / "AVL" / "B-Tree") together with the *dominant* shape type.

Scope and validation

Validated against the demo deck: the AVL example slide reconstructs as three distinct trees; the B-Tree insert slides reconstruct as six and three insertion states respectively, matching the drawn sequences.

In scope now: insertion for binary/AVL trees (static) and B-trees (animated). Not yet handled: deletion (shapes disappearing and restructuring), AVL rotation (motion-driven restructuring rather than appearance), and heaps/graphs. Key extraction is best-effort on hand-authored slides that overlap or reuse shapes, so generated descriptions should be checked against the source — a sighted-evaluator verification pass before any user study.